Complete Guide to JPA Relationships

Table of Contents

1. Relationship Fundamentals
2. One-to-One Relationships
3. One-to-Many / Many-to-One Relationships
4. Many-to-Many Relationships
5. Important Annotations Reference
6. Best Practices & Common Patterns

Relationship Fundamentals

Key Concepts

Owning Side vs Inverse Side:

- **Owning Side**: Contains the foreign key in the database (uses `@JoinColumn`)
- **Inverse Side**: References the owning side (uses `mappedBy`)
- Only the owning side's changes are persisted to the database

Unidirectional vs Bidirectional:

- **Unidirectional**: Navigation in one direction only (A → B)
- **Bidirectional**: Navigation in both directions (A ↔ B)

Important Annotations:

- `@OneToOne`, `@OneToMany`, `@ManyToOne`, `@ManyToMany`
- `@JoinColumn`: Specifies the foreign key column
- `@JoinTable`: Defines the join table for many-to-many relationships
- `mappedBy`: Marks the inverse side of a bidirectional relationship

---

One-to-One Relationships

A one-to-one relationship means one entity instance is associated with exactly one instance of another entity.

1.1 Unidirectional One-to-One (Foreign Key in Source)

Scenario: Each Employee has one ParkingSpace, but ParkingSpace doesn't know about Employee.

Database Structure:

```sql
CREATE TABLE employee (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255),
    parking_space_id BIGINT UNIQUE,  -- Foreign key
    FOREIGN KEY (parking_space_id) REFERENCES parking_space(id)
);

CREATE TABLE parking_space (
    id BIGINT PRIMARY KEY,
    space_number VARCHAR(10)
);
```

Java Code:

```java
@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "parking_space_id", referencedColumnName = "id")
    private ParkingSpace parkingSpace;

    // Constructors, getters, setters
}

@Entity
public class ParkingSpace {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    private String spaceNumber;

    // No reference to Employee
    // Constructors, getters, setters
}
```

Key Annotations:

- `@OneToOne` : Defines the relationship
- `@JoinColumn(name = "parking_space_id")` : Specifies the foreign key column name
- `referencedColumnName = "id"` : References the primary key in ParkingSpace (optional if referencing primary key)

1.2 Bidirectional One-to-One (Foreign Key in Source)

Scenario: Employee has ParkingSpace, and ParkingSpace knows which Employee owns it.

Database Structure: Same as 1.1 (foreign key still in employee table)

Java Code:

```
@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    // Owning side (has @JoinColumn)
    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "parking_space_id", referencedColumnName = "id")
    private ParkingSpace parkingSpace;

    // Getters, setters
    public void setParkingSpace(ParkingSpace parkingSpace) {
        this.parkingSpace = parkingSpace;
        if (parkingSpace != null) {
            parkingSpace.setEmployee(this);  // Sync both sides
        }
    }
```

```
}

@Entity
public class ParkingSpace {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String spaceNumber;

    // Inverse side (uses mappedBy)
    @OneToOne(mappedBy = "parkingSpace")
    private Employee employee;

    // Getters, setters
    public void setEmployee(Employee employee) {
        this.employee = employee;
    }
}
```

Key Points:

- `mappedBy = "parkingSpace"` : Points to the field name in Employee class
- The inverse side doesn't control the relationship
- Always sync both sides in setter methods

1.3 Bidirectional One-to-One (Foreign Key in Target)

Scenario: Same as 1.2, but foreign key is in the parking_space table.

Database Structure:

```
CREATE TABLE employee (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255)
);

CREATE TABLE parking_space (
    id BIGINT PRIMARY KEY,
    space_number VARCHAR(10),
```

```
    employee_id BIGINT UNIQUE,  -- Foreign key here
    FOREIGN KEY (employee_id) REFERENCES employee(id)
);
```

Java Code:

```
@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    // Inverse side
    @OneToOne(mappedBy = "employee", cascade = CascadeType.ALL)
    private ParkingSpace parkingSpace;

    // Helper method
    public void setParkingSpace(ParkingSpace parkingSpace) {
        this.parkingSpace = parkingSpace;
        if (parkingSpace != null) {
            parkingSpace.setEmployee(this);
        }
    }
}

@Entity
public class ParkingSpace {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String spaceNumber;

    // Owning side
    @OneToOne
    @JoinColumn(name = "employee_id", referencedColumnName = "id")
    private Employee employee;
```

```
    // Getters, setters
}
```

1.4 One-to-One with Shared Primary Key

Scenario: Employee and EmployeeDetails share the same primary key.

Database Structure:

```
CREATE TABLE employee (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255)
);

CREATE TABLE employee_details (
    id BIGINT PRIMARY KEY,  -- Same value as employee.id
    bio TEXT,
    hire_date DATE,
    FOREIGN KEY (id) REFERENCES employee(id)
);
```

Java Code:

```
@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToOne(mappedBy = "employee", cascade = CascadeType.ALL)
    private EmployeeDetails details;

    // Getters, setters
}

@Entity
public class EmployeeDetails {
```

```
    @Id
    private Long id;  // No @GeneratedValue!

    private String bio;
    private LocalDate hireDate;

    @OneToOne
    @MapsId  // Use employee's ID as this entity's ID
    @JoinColumn(name = "id")
    private Employee employee;

    // Getters, setters
}
```

Key Annotation:

- `@MapsId` : Makes this entity use the ID from the associated entity

---

1.5 One-to-One with Optional Relationship

Scenario: Employee may or may not have a ParkingSpace.

```
@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToOne(cascade = CascadeType.ALL, optional = true)
    @JoinColumn(name = "parking_space_id", nullable = true)
    private ParkingSpace parkingSpace;  // Can be null

    // Getters, setters
}
```

Key Attributes:

- `optional = true` : Relationship is optional (default is true)

- `nullable = true` : Column can be null in database

One-to-Many / Many-to-One Relationships

The most common relationship type in JPA.

2.1 Unidirectional Many-to-One

Scenario: Many Employees belong to one Department, but Department doesn't track its employees.

Database Structure:

```sql
CREATE TABLE department (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255)
);

CREATE TABLE employee (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255),
    department_id BIGINT,  -- Foreign key
    FOREIGN KEY (department_id) REFERENCES department(id)
);
```

Java Code:

```java
@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id", nullable = false)
    private Department department;
```

```
    // Getters, setters
}

@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    // No reference to employees
    // Getters, setters
}
```

Key Points:

- `@ManyToOne` : Many employees to one department
- Always use `FetchType.LAZY` for better performance
- The many side is always the owning side in @ManyToOne

2.2 Bidirectional One-to-Many / Many-to-One (Most Common)

Scenario: Department has many Employees, and each Employee knows its Department.

Database Structure: Same as 2.1

Java Code:

```
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    // Inverse side (uses mappedBy)
    @OneToMany(
        mappedBy = "department",           // Field name in Employee
```

```
        cascade = CascadeType.ALL,
        orphanRemoval = true,
        fetch = FetchType.LAZY
    )
    private List<Employee> employees = new ArrayList<>();

    // Helper methods for bidirectional sync
    public void addEmployee(Employee employee) {
        employees.add(employee);
        employee.setDepartment(this);
    }

    public void removeEmployee(Employee employee) {
        employees.remove(employee);
        employee.setDepartment(null);
    }

    // Getters, setters
}

@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    // Owning side (has @JoinColumn)
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id", nullable = false)
    private Department department;

    // Getters, setters
    public void setDepartment(Department department) {
        this.department = department;
    }
}
```

Key Annotations Explained:

- `mappedBy = "department"` : Points to the field name in Employee

- `cascade = CascadeType.ALL` : Operations on Department cascade to Employees
- `orphanRemoval = true` : If Employee is removed from collection, it's deleted from DB
- **Important**: Always use helper methods to keep both sides synchronized!

2.3 Unidirectional One-to-Many with Join Table

Scenario: Department tracks Employees, but Employee doesn't reference Department. Uses a join table instead of foreign key in employee table.

Database Structure:

```
CREATE TABLE department (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255)
);

CREATE TABLE employee (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255)
    -- No department_id column!
);

CREATE TABLE department_employees (
    department_id BIGINT,
    employee_id BIGINT,
    PRIMARY KEY (department_id, employee_id),
    FOREIGN KEY (department_id) REFERENCES department(id),
    FOREIGN KEY (employee_id) REFERENCES employee(id)
);
```

Java Code:

```
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
```

```
    @OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
    @JoinTable(
        name = "department_employees",
        joinColumns = @JoinColumn(name = "department_id"),
        inverseJoinColumns = @JoinColumn(name = "employee_id")
    )
    private List<Employee> employees = new ArrayList<>();

    // Getters, setters
}

@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    // No reference to Department
    // Getters, setters
}
```

Key Annotation:

- `@JoinTable` : Creates an intermediate join table
- `joinColumns` : Column referring to the owning entity (Department)
- `inverseJoinColumns` : Column referring to the target entity (Employee)

Note: This approach is less common and less efficient than bidirectional Many-to-One. Use only when you truly need unidirectional one-to-many.

2.4 One-to-Many with Composite Key

Scenario: OrderLine has a composite key (orderId + productId).

```
@Embeddable
public class OrderLineId implements Serializable {
    private Long orderId;
```

```java
    private Long productId;

    // Default constructor
    public OrderLineId() {}

    public OrderLineId(Long orderId, Long productId) {
        this.orderId = orderId;
        this.productId = productId;
    }

    // equals() and hashCode() are REQUIRED
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        OrderLineId that = (OrderLineId) o;
        return Objects.equals(orderId, that.orderId) &&
               Objects.equals(productId, that.productId);
    }

    @Override
    public int hashCode() {
        return Objects.hash(orderId, productId);
    }

    // Getters, setters
}

@Entity
public class Order {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL)
    private List<OrderLine> orderLines = new ArrayList<>();

    // Getters, setters, helper methods
}

@Entity
public class OrderLine {
```

```
    @EmbeddedId
    private OrderLineId id;

    @ManyToOne
    @MapsId("orderId")  // Maps to orderId in OrderLineId
    @JoinColumn(name = "order_id")
    private Order order;

    @ManyToOne
    @MapsId("productId")  // Maps to productId in OrderLineId
    @JoinColumn(name = "product_id")
    private Product product;

    private Integer quantity;

    // Getters, setters
}
```

2.5 One-to-Many with @OrderColumn

Scenario: Maintain the order of elements in a list.

```
@Entity
public class Playlist {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToMany(mappedBy = "playlist", cascade = CascadeType.ALL)
    @OrderColumn(name = "position")  // Adds position column
    private List<Song> songs = new ArrayList<>();

    // Getters, setters
}

@Entity
public class Song {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;

    @ManyToOne
    @JoinColumn(name = "playlist_id")
    private Playlist playlist;

    // Getters, setters
}
```

Database Structure:

```
CREATE TABLE song (
    id BIGINT PRIMARY KEY,
    title VARCHAR(255),
    playlist_id BIGINT,
    position INTEGER,  -- Order column
    FOREIGN KEY (playlist_id) REFERENCES playlist(id)
);
```

2.6 One-to-Many with @OrderBy

Scenario: Sort collection by a specific field when loading.

```
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToMany(mappedBy = "department")
    @OrderBy("lastName ASC, firstName ASC")  // SQL ORDER BY clause
    private List<Employee> employees = new ArrayList<>();
```

```
    // Getters, setters
}
```

Key Difference:

- `@OrderColumn` : Maintains a specific order (requires extra column)
- `@OrderBy` : Sorts by existing fields (no extra column needed)

Many-to-Many Relationships

Two entities can have multiple instances of each other.

3.1 Unidirectional Many-to-Many

Scenario: Employees can work on multiple Tasks, but Task doesn't track its employees.

Database Structure:

```sql
CREATE TABLE employee (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255)
);

CREATE TABLE task (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255)
);

CREATE TABLE employee_task (
    employee_id BIGINT,
    task_id BIGINT,
    PRIMARY KEY (employee_id, task_id),
    FOREIGN KEY (employee_id) REFERENCES employee(id),
    FOREIGN KEY (task_id) REFERENCES task(id)
);
```

Java Code:

```java
@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
    @JoinTable(
        name = "employee_task",
        joinColumns = @JoinColumn(name = "employee_id"),
        inverseJoinColumns = @JoinColumn(name = "task_id")
    )
    private List<Task> tasks = new ArrayList<>();

    // Getters, setters
}

@Entity
public class Task {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    // No reference to Employee
    // Getters, setters
}
```

Key Points:

- `@ManyToMany` : Defines many-to-many relationship
- `@JoinTable` : Required for many-to-many (creates join table)
- **Avoid** `CascadeType.REMOVE` : Deleting one employee shouldn't delete all their tasks!

3.2 Bidirectional Many-to-Many (Most Common)

Scenario: Employees work on Tasks, and Tasks track their employees.

Database Structure: Same as 3.1

Java Code:

```
@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    // Owning side (defines @JoinTable)
    @ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
    @JoinTable(
        name = "employee_task",
        joinColumns = @JoinColumn(name = "employee_id"),
        inverseJoinColumns = @JoinColumn(name = "task_id")
    )
    private List<Task> tasks = new ArrayList<>();

    // Helper methods to keep both sides in sync
    public void addTask(Task task) {

        tasks.add(task);
        task.getEmployees().add(this);
    }

    public void removeTask(Task task) {
        tasks.remove(task);
        task.getEmployees().remove(this);
    }

    // Getters, setters
}

@Entity
public class Task {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;

    private String name;

    // Inverse side (uses mappedBy)
    @ManyToMany(mappedBy = "tasks")
    private List<Employee> employees = new ArrayList<>();

    // Helper methods
    public void addEmployee(Employee employee) {
        employees.add(employee);
        employee.getTasks().add(this);
    }

    public void removeEmployee(Employee employee) {
        employees.remove(employee);
        employee.getTasks().remove(this);
    }

    // Getters, setters
}
```

Important Notes:

- Always use helper methods to synchronize both sides
- Implement `equals()` and `hashCode()` based on business key (not ID)
- Never use `CascadeType.REMOVE` in many-to-many
- The side with `@JoinTable` is the owning side

3.3 Many-to-Many with Additional Attributes

Problem: What if you need extra data in the join table (e.g., assignment date, role)?

Solution: Create an intermediate entity!

Scenario: Employee works on Projects with a specific role and start date.

Database Structure:

```
CREATE TABLE employee (
    id BIGINT PRIMARY KEY,
```

```sql
    name VARCHAR(255)
);

CREATE TABLE project (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255)
);

CREATE TABLE project_assignment (
    id BIGINT PRIMARY KEY,  -- Now has its own ID
    employee_id BIGINT,
    project_id BIGINT,
    role VARCHAR(50),       -- Additional attribute
    start_date DATE,        -- Additional attribute
    FOREIGN KEY (employee_id) REFERENCES employee(id),
    FOREIGN KEY (project_id) REFERENCES project(id)
);
```

Java Code:

```java
@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToMany(mappedBy = "employee", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<ProjectAssignment> projectAssignments = new ArrayList<>();

    // Helper method
    public void assignToProject(Project project, String role, LocalDate startDate) {
        ProjectAssignment assignment = new ProjectAssignment(this, project, role, startDate);
        projectAssignments.add(assignment);
        project.getProjectAssignments().add(assignment);
    }

    // Getters, setters
}
```

```java
@Entity
public class Project {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToMany(mappedBy = "project", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<ProjectAssignment> projectAssignments = new ArrayList<>();

    // Getters, setters
}

@Entity
@Table(name = "project_assignment")
public class ProjectAssignment {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "employee_id")
    private Employee employee;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "project_id")
    private Project project;

    private String role;           // Additional attribute
    private LocalDate startDate;   // Additional attribute

    // Constructor
    public ProjectAssignment() {}

    public ProjectAssignment(Employee employee, Project project, String role, LocalDate startDate) {
        this.employee = employee;
        this.project = project;
        this.role = role;
        this.startDate = startDate;
    }
```

```
    // Getters, setters

    // Important: equals() and hashCode()
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        ProjectAssignment that = (ProjectAssignment) o;
        return Objects.equals(employee, that.employee) &&
               Objects.equals(project, that.project);
    }

    @Override
    public int hashCode() {
        return Objects.hash(employee, project);
    }
}
```

Key Pattern: Convert Many-to-Many to two One-to-Many relationships with an intermediate entity.

3.4 Many-to-Many with Composite Key (Alternative Approach)

Same scenario as 3.3, but using composite key instead of separate ID.

```
@Embeddable
public class ProjectAssignmentId implements Serializable {
    private Long employeeId;
    private Long projectId;

    // Default constructor
    public ProjectAssignmentId() {}

    public ProjectAssignmentId(Long employeeId, Long projectId) {
        this.employeeId = employeeId;
        this.projectId = projectId;
    }

    // equals() and hashCode() required
    @Override
    public boolean equals(Object o) {
```

```java
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        ProjectAssignmentId that = (ProjectAssignmentId) o;
        return Objects.equals(employeeId, that.employeeId) &&
               Objects.equals(projectId, that.projectId);
    }

    @Override
    public int hashCode() {
        return Objects.hash(employeeId, projectId);
    }

    // Getters, setters
}

@Entity
@Table(name = "project_assignment")
public class ProjectAssignment {
    @EmbeddedId
    private ProjectAssignmentId id;

    @ManyToOne
    @MapsId("employeeId")
    @JoinColumn(name = "employee_id")
    private Employee employee;

    @ManyToOne
    @MapsId("projectId")
    @JoinColumn(name = "project_id")
    private Project project;

    private String role;
    private LocalDate startDate;

    // Getters, setters
}
```

3.5 Many-to-Many with Set Instead of List

Better performance and avoids duplicates.

```java
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private Set<Course> courses = new HashSet<>();

    // Helper methods
    public void enrollInCourse(Course course) {
        courses.add(course);
        course.getStudents().add(this);
    }

    public void dropCourse(Course course) {
        courses.remove(course);
        course.getStudents().remove(this);
    }

    // IMPORTANT: Override equals() and hashCode()
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Student student = (Student) o;
        return Objects.equals(id, student.id);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
```

```
    // Getters, setters
}

@Entity
public class Course {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToMany(mappedBy = "courses")
    private Set<Student> students = new HashSet<>();

    // equals(), hashCode(), getters, setters
}
```

Why Set?

- No duplicates allowed
- Better performance for contains() checks
- More semantically correct for many-to-many

Important: You MUST override `equals()` and `hashCode()` when using Set!

Important Annotations Reference

@JoinColumn

Specifies the foreign key column.

```
@JoinColumn(
    name = "department_id",              // Column name in this table
    referencedColumnName = "id",         // Column name in target table (default: PK)
    nullable = false,                    // NOT NULL constraint
    unique = false,                      // UNIQUE constraint
    insertable = true,                   // Include in INSERT
    updatable = true,                    // Include in UPDATE
    columnDefinition = "BIGINT DEFAULT 0", // Custom SQL definition
```

```
    table = "employee",                  // Table name (for secondary tables)
    foreignKey = @ForeignKey(name = "fk_emp_dept") // FK constraint name
)
private Department department;
```

@JoinTable

Defines the join table for many-to-many relationships.

```
@JoinTable(
    name = "employee_task",              // Join table name
    joinColumns = @JoinColumn(
        name = "employee_id",            // FK to owning entity
        referencedColumnName = "id"
    ),
    inverseJoinColumns = @JoinColumn(
        name = "task_id",                // FK to target entity
        referencedColumnName = "id"
    ),
    uniqueConstraints = @UniqueConstraint(
        columnNames = {"employee_id", "task_id"}
    ),
    indexes = {
        @Index(name = "idx_employee", columnList = "employee_id"),
        @Index(name = "idx_task", columnList = "task_id")
    }
)
private List<Task> tasks;
```

---

mappedBy

Marks the inverse side of a bidirectional relationship.

```
// CORRECT ✓
@Entity
public class Department {
```

```
    @OneToMany(mappedBy = "department")  // Points to field name in Employee
    private List<Employee> employees;
}

@Entity
public class Employee {
    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;  // This field name must match mappedBy
}

// INCORRECT ✗
@Entity
public class Department {
    @OneToMany(mappedBy = "dept")  // Wrong! Field is called "department", not "dept"
    private List<Employee> employees;
}
```

Rules:

- Value must match the field name in the owning entity
- Only on the inverse side (never with @JoinColumn)
- Cannot be used with @JoinTable

Cascade Types

```
@OneToMany(cascade = {
    CascadeType.PERSIST,   // persist() cascades to children
    CascadeType.MERGE,     // merge() cascades to children
    CascadeType.REMOVE,    // remove() cascades to children
    CascadeType.REFRESH,   // refresh() cascades to children
    CascadeType.DETACH,    // detach() cascades to children
    CascadeType.ALL        // All of the above
})
```

Best Practices:

- **One-to-One:** `CascadeType.ALL` is usually fine
- **One-to-Many:** `CascadeType.ALL` with `orphanRemoval = true`

- **Many-to-One:** `{PERSIST, MERGE}` only
- **Many-to-Many:** `{PERSIST, MERGE}` only (never REMOVE!)

Fetch Types

```
// LAZY: Load when accessed (default for collections)
@OneToMany(fetch = FetchType.LAZY)
private List<Employee> employees;

// EAGER: Load immediately (default for single-valued)
@ManyToOne(fetch = FetchType.EAGER)
private Department department;
```

Defaults:

- `@OneToOne` : EAGER
- `@ManyToOne` : EAGER
- `@OneToMany` : LAZY
- `@ManyToMany` : LAZY

Best Practice: Always explicitly set to LAZY, override with fetch joins when needed.

---

orphanRemoval

Automatically delete entities removed from a collection.

```
@Entity
public class Department {
    @OneToMany(
        mappedBy = "department",
        cascade = CascadeType.ALL,
        orphanRemoval = true  // Delete orphaned employees
    )
    private List<Employee> employees = new ArrayList<>();
}

// Usage
Department dept = em.find(Department.class, 1L);
```

```
Employee emp = dept.getEmployees().get(0);
dept.getEmployees().remove(emp);  // emp is deleted from database!

// vs without orphanRemoval
dept.getEmployees().remove(emp);  // emp remains in database with null dept
```

When to use:

- One-to-Many where child can't exist without parent
- One-to-One with dependent entity

When NOT to use:

- Many-to-Many (would delete shared entities!)
- Many-to-One (doesn't make sense)

Best Practices & Common Patterns

1. Always Synchronize Bidirectional Relationships

❌ **BAD:**

```
department.getEmployees().add(employee);  // Only one side updated!
```

✅ **GOOD:**

```
// Use helper methods
public class Department {
    public void addEmployee(Employee employee) {
        employees.add(employee);
        employee.setDepartment(this);  // Sync both sides
    }

    public void removeEmployee(Employee employee) {
        employees.remove(employee);
        employee.setDepartment(null);  // Sync both sides
    }
}
```

```
// Usage
department.addEmployee(employee);  // Both sides synced
```

---

2. Implement equals() and hashCode() Properly

For entities in collections (especially with Set):

```
@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true)
    private String email;  // Natural business key

    // Use business key, NOT id
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Employee employee = (Employee) o;
        return Objects.equals(email, employee.email);
    }

    @Override
    public int hashCode() {
        return Objects.hash(email);
    }
}
```

Why not use ID?

- ID is null before persist()
- Causes issues when adding to Set before persisting

---

3. Initialize Collections

✅ **Always initialize collections:**

```
@Entity
public class Department {
    @OneToMany(mappedBy = "department")
    private List<Employee> employees = new ArrayList<>();  // ✓ Initialized
}
```

❌ **Don't leave null:**

```
@Entity
public class Department {
    @OneToMany(mappedBy = "department")
    private List<Employee> employees;  // ✗ NullPointerException waiting to happen
}
```

---

4. Use Lazy Loading by Default

❌ **BAD:**

```
@ManyToOne(fetch = FetchType.EAGER)
private Department department;  // Loads department every time!
```

✅ **GOOD:**

```
@ManyToOne(fetch = FetchType.LAZY)
private Department department;

// Use fetch joins when needed
String jpql = "SELECT e FROM Employee e JOIN FETCH e.department WHERE e.id = :id";
```

---

5. Be Careful with Cascade Types

❌ **BAD - Many-to-Many with REMOVE:**

```
@ManyToMany(cascade = CascadeType.ALL)  // Includes REMOVE!
private List<Task> tasks;

// Deleting employee deletes all tasks, even if other employees need them!
em.remove(employee);
```

✅ **GOOD:**

```
@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
private List<Task> tasks;
```

---

6. Avoid Unidirectional OneToMany with Join Table

❌ **Less efficient:**

```
@Entity
public class Department {
    @OneToMany
    @JoinTable(...)
    private List<Employee> employees;
}
```

✅ **Better - Use bidirectional with ManyToOne:**

```
@Entity
public class Department {
    @OneToMany(mappedBy = "department")
    private List<Employee> employees;
}

@Entity
public class Employee {
    @ManyToOne
    @JoinColumn(name = "department_id")
```

```
    private Department department;
}
```

---

7. toString() Pitfall

❌ **BAD - Causes infinite loop:**

```
@Entity
public class Employee {
    @ManyToOne
    private Department department;

    @Override
    public String toString() {
        return "Employee{department=" + department + "}";  // Calls department.toString()
    }
}

@Entity
public class Department {
    @OneToMany(mappedBy = "department")
    private List<Employee> employees;

    @Override
    public String toString() {
        return "Department{employees=" + employees + "}";  // Calls employees.toString() → StackOverflow!
    }
}
```

✅ **GOOD:**

```
@Override
public String toString() {
    return "Employee{id=" + id + ", name=" + name + "}";  // Don't include relationships
}
```

---

8. Use @MapsId for One-to-One with Shared PK

More efficient than separate foreign key:

```
@Entity
public class User {
    @Id
    @GeneratedValue
    private Long id;

    @OneToOne(mappedBy = "user", cascade = CascadeType.ALL)
    private UserProfile profile;
}

@Entity
public class UserProfile {
    @Id
    private Long id;  // Same as user.id

    @OneToOne
    @MapsId
    @JoinColumn(name = "id")
    private User user;
}
```

Summary Table

Relationship	Unidirectional	Bidirectional	Join Table?	Common Use Case
One-to-One	✓	✓	Optional	Employee ↔ ParkingSpace
Many-to-One	✓	N/A	No	Employee → Department
One-to-Many	✓ (rare)	✓	Optional	Department ↔ Employee
Many-to-Many	✓	✓	Yes	Employee ↔ Task

Quick Decision Guide

Choose relationship type:

1. Can A have multiple B? Can B have multiple A?

- Yes + Yes = **Many-to-Many**
- Yes + No = **One-to-Many**
- No + No = **One-to-One**
- No + Yes = **Many-to-One**

Choose directionality:

- Need navigation both ways? → **Bidirectional**
- Navigate one way only? → **Unidirectional**

Choose owning side:

- **One-to-Many/Many-to-One**: Many side is always owning
- **One-to-One**: Side with foreign key is owning
- **Many-to-Many**: Side with @JoinTable is owning

Related Guides:

- `jpa-guide.md` : Complete JPA reference
- `queries.md` : JPQL queries
- `task.md` : Project requirements

Last Updated: 2024